

Presentation Transcript

سلام عليكم ورحمة الله وبركاته، أنا مصطفى أحمد ودلوقتي هنتكلم علي جزء الـ HW/SW co-design بالتحديد integration الـ butterfly للـ SW FFT.

Slide 1: Problem Statement

الـ objective من جزء الـ task الأول هو الـ implementing الـ RTL بتاع الـ butterfly، عملت الـ implementation لـ radix 2 butterfly ونعمله integration كـ AXI peripheral في الـ pulp platform ونعمله drive من الـ SW fft.

الهدف من الـ task هو اننا نـ validate الـ hw, sw interaction ونشوف هل فعلا حصل الـ Acceleration.

الـ integration بتنقسم لجزئين: الأول الـ HW integration، هنا بكتب الـ rtl ونوصف الـ ports بالـ hjson عشان نـ generate الـ register file الـ sw والـ hw هيتعمله علي اساسه، ونـ design الـ protocol translation wrapper ونعمل الـ C test بسيط عشان اتأكد إن الـ butterfly visible قبل ما نعمله drive من الـ FFT.

ونجحن في عملية الـ integration وعملية الـ end to end hw/sw co design.

Slide: Pulp SoC architecture

اولا هنكلم overview عن الـ pulp architecture عشان نعرف ايه الـ steps الـ محتاجين نعملها في عملية الـ integration.

في العاده الـ processor الـ هو CPU بيعتمد علي الـ caches، لو محتاج الـ data ببشوفها في الـ cache لو مش موجودة الـ hw ببـ fetch الـ data من الـ main memory للـ cache وتم العملية. فده في الـ unpredictability لأن لو حصل الـ hit هتكون الـ execution سريعة لو حصل الـ miss الـ execution هتكون ابطأ لأن هيجصل الـ stall.

الـ pulp platform مش بيتبع الطريقة دي للـ data memory هي بتستخدم الـ TCDM الـ هي الـ tightly coupled data memory، وهي عبارة عن الـ memory mapped address space بيتحكم فيها الـ sw. ف الـ concept الـ miss مش موجود عشان عن طريق الـ sw او الـ DMA engine بيودي الـ data للـ TCDM عمدا، فلما الـ CPU او الـ HW ببقرأ من الـ TCDM بيبقي ضامن ان الـ data موجودة لأن الـ SW جابها فبيكون الـ behavior predictable.

بيكون في 3 main masters عايزين يعملهم الـ Read او الـ write للـ data:

- اول واحد هو الـ fabric controller وهو الـ RISC processor الـ بيرن الـ c code ويـ setup الـ operations ويمكن ينقل الـ data للـ TCDM.
- ثاني واحد هو الـ micro DMA وده ممكن نقول عليه delivery service لأنه بينقل الـ data الجاية للـ TCDM عطلول من غير إحتياج الـ processor.
- والثالث هو اي الـ accelerator وهو الـ Hw الـ مكتوب عشان يعمل الـ function محددة.

يجي هنا سؤال: طب لو الـ 3 عايزين يـ access الـ TCDM في نفس الوقت ايه الـ هيجصل؟ اكيد الـ Collision، عشان كده بنحتاج الـ traffic router عشان يـ handle الدنيا وده هو الـ SoC interconnect.

Slide: Interconnect and Arbitration

فزي ما قولنا محتاجين نفهم الـ interconnect شغال ازاى.

هو بينقسم لإثنين networks:

الأول TCDM interconnect وده fast lane و predictable زي ما قولنا الـ reads بتاخذ 1 clock cycle. وهو سريع عشان هو مش block memory كبير لا هو اكتر من parallel memory bank الـ هو interleaved memory فمثلاً لو الـ micro DMA بتكتب في bank 0 الـ fabric controller ممكن يكون بيقرأ من bank 1 من غير ما يبطأ.

والثاني هو الـ AXI crossbar وده بيستخدم للحاجات اللي بتحتاج delay أو operations معقدة. بس في تصميمنا، إحنا مش بنوقف الـ bus لحد ما الـ hardware يخلص! الـ processor بيكتب الـ twiddle index والـ operands في الـ registers، والـ AXI transaction بتخلص فوراً والـ bus بيفضى لأي حد تاني. في الوقت ده الـ hardware بتاعنا بيـ read الـ twiddle factors من الـ internal ROM وبيحسب مع نفسه، والـ processor بيعمل polling للـ status لحد ما النتيجة تجهز.

الـ processor بيكلم الـ hardware عن طريق memory address، الـ hw ملوش اسم يعني هو عبارة عن الـ address ده لو اتكتب فيه الـ interconnect يـ route للـ butterfly.

فالـ communication ده بين الـ HW/SW بيتقسم لإثنين paths: الـ control path ده عن طريقة بينكتب الـ configurations في الـ Memory mapped registers الـ automatically generated بناءً علي التوصيف الـ اتكتب في الـ hjson الـ path ده مثلاً للـ CTRL او الـ STATUS bits.

الـ path الثاني هو الـ Data path ده الـ بيعت الـ heavy data زي الـ operands علي الـ axi bus للـ registers.

حالياً احنا بنبتع الـ data عن طريق الـ sw ولكن ممكن يستخدم الـ DMA عشان بيعت الـ data عشان منعطلش الـ processor.

تقابلنا مشكلة كمان الـ AXI protocol معقد والـ RTL مش بيّفهمه فلزام يكون في translation layer كده بين الـ C pointers بتاعت الـ RISC والـ hw physical wires.

Slide: The Translation Boundary

فأول حاجة في الـ Translation بنعملها قبل الـ wrapper اننا بنوصف الـ registers الـ احنا محتاجينها وهل هما read only ولا write only ومن اني ناحية يعني hwro او swro.

بنعمل الـ hjson ده الـ هو العقد بين الـ hw والـ sw عشان صعب جدا نكتب الـ address decoding ده manually فبنـ pass الـ hjson ده لـ reggen tool وهي EDA tool بتـ generate system verilog register file بيوصف هو الكلام ده.

تمام وصفنا الكلام ده محتاجين دلوقتي نبدأ عملية protocol downgrade.

الواصل الـ wrapper الـ instantiated في pulp soc domain هو axi transaction فبنحتاج protocol converter يحولها لـ REG_BUS أبسط عن طريق انه يبشيل الـ AXI overhead وبيفضل interface أبسط write data, read address, flags, data, (axi_to_reg_intf) كده.

بعدها بنستخدم macros بتحول الـ REG_BUS system verilog interface ده لـ structs زي (reg_req_t) و (reg_rsp_t).

طب ليه structs؟ عشان ال register file generated بال reggen مش بيد support ال system verilog interface ويد support ال structs.

ال register file بياخد ال data الممثلة في ال structs ويتعامل معاهم ك hw wires:

- q wire: بيحتفظ بال operands stable عشان ال hw يقرأهم.
- we wire: ده write enable بيد trigger لو one cycle عشان نبدأ ال operation.
- d wire: ده data wire بيتحتط عليه ال Results.
- de wire: ده data enable لما بيق high معناها روح بص علي ال data wire لو low متبصش.

Slide: SoC Modifications

آخر حاجة مطلوب نعملها هو اننا نوصل ال axi peripheral ال زودناه لل interconnect عشان بيق شافه، فلانم ن extended crossbar عشان نضيف Axi slave كمان.

بنحدد ال Address region ال بتعبر عن ال peripheral ده فبنعرف START و END addresses في ال .soc_mem_map.svh

لازم نضمن ان مفيش memory overlap ون specify memory كفاية انا خليتها 4K bytes.

بعدها نضيف addressing rule عشان نعرف ال crossbar يعمل ايه لو جاله operation متعلقة بال Address region دي، وبكده ال peripheral الجديد بقي visible لل bus.

Slide: Butterfly Operation

نتكلم بقي عن ال operation ال butterfly بسرعة هما كانوا اتكلموا عنها في الأول.

Radix butterfly بيكون في sum path و diff path ال diff بيضرب في twiddle factors يعني بيحصله rotation.

فهنا ده ال heavy computation في ال FFT وبيستخدم تقريبا في اغلب ال stages عشان كده ال بيحصله offloading من ال processor ويتعمله unit مخصوص.

Slide: RTL Design

ال RTL design عملت ٢ butterflies او نقدر نقول ٣ اول واحد كان بيستقبل ال twiddle factors ك inputs وبيستخدم pipelined multiplier و multiplier width كان بيتحدد dynamically حسب ال operand العنده bits اقل لإن complexity 24x 12 multiplier بتكون زي 12 multiplier x 24.

بس مش ده النهائي ال شغال بيه حاليا هو standard multiplier عشان بيق زي ال implemented في ال fft فال fair comparison تبقي وبستقبل twiddle index و read ال twiddle factor من internal ROM هنتكلم علي النقطة دي قدام.

ال diagram ده بيوضح ال data flow في ال rtl.

(اشرح ال diagram)

Slide: Arithmetic Optimization

الـ complex multiplication هو اعقد عملية بتحصل في الـ datapath بتاعنا الـ standard twiddle multiplication هتحتاج 4 real multipliers وده يستهلك area اكتر و power.

فعملنا reduction من 4 multipliers لـ 3 بمجرد reorganization وده هو الـ gauss trick الـ هي kartsuba optimization. فعدد الـ multipliers قل فالـ area قلت والـ power في المقابل عدد الـ adders والـ Subtractors زاد ولكن المكسب اكتر.

بيتم عن طريق اننا بنحسب 3 intermediate products بعدها بنعمل construction للـ results.

زي الـ figure ما هو مبين بنضرب الـ real parts في بعض والـ imaginary في بعض والـ product الثالث بنجمع real و imag بتاع operand و real و imag بتاع الـ Twiddle ونضربه. الـ real part بيكون الـ product الاول minus الـ ثاني والـ Imag part بيكون الـ $p_3 \text{ minus } p_2 \text{ minus } p_1$.

Slide: Pipeline Alignment

بما ان الـ diff path بيضرب في twiddle فالـ Sum path هيكون اسرع منه عشان الوقت الـ multiplier بياخده فبنعمل delaying للـ Sum path عشان الـ results تكون aligned اول ما يخلص بنرفع out flag يـ indicate ان الـ computation خلص.

Slide: Pulp Integration

دي الـ steps الـ integration للـ butterfly الـ اتكلمنا عنها في الأول بشكل general.

Slide: Wrapper Architecture

وده الـ wrapper architecture بتاع الـ butterfly برضو زي الجزء الـ في الأول الـ data flow.

الـ Processor بيعت الـ axi transaction بعدها نحولها لـ reg_bus protocol بالـ protocol converter بعدها بالـ macros نحوله لـ system verilog structs عشان ده الـ supported بعدها الـ register file الـ butterfly يقرأ ويكتب فيه.

Slide: Software Verification 1

عايزين نـ test بقي فعلت driver بسيط جدا عشان اشوف شغال صح والـ butterfly visible و integrated ولا لا فكان الـ butterfly مطبوط والـ Results متطابقة مع الـ testbench.

Slide: Optimization 1 Auto Trigger

بعد ما اتأكدت من الـ integration والـ operation حسبت الـ butterfly الـ latency بتاعته قد ايه فكانت cycle 442 فحاجة مش طبيعية، ده كان للـ butterfly الـ يستخدم pipelined multiplier وبيتبع الـ twiddle له مش index بس ومش معمول فيه الـ optimizations دي.

(Note: الـ 442 كان sizes الـ registers فيها كبير مش الـ 12 input والـ 12 output فكانت defined multireg في الـ hjson فده وقت اطول فالكتابة و axi transactions اكتر لأنه بيعت 32 bit بس كل transaction).

فأول optimization عملته هو الـ auto trigger فيبدل ما الـ sw يكتب الـ left والـ right operands وكماني يكتب الـ trigger في separate ctrl register، خليفته auto triggered بعد ما الـ operand right يكتب عطل الـ signal qe تبقى high 1 cycle ويبدأ الـ computation.

فكده وفرت axi transaction والـ latency قلت.

Slide: Optimization 2 Auto Status

كنا برضو بنستهلك transaction عشاني set او clear الـ STATUS.

ففي الـ wrapper خليت الموضوع ده automatically فبعد ما الـ right operand يكتب automatically الـ status بيبقي 0 الـ هو busy.

بحيث الـ data Enable يطلع high فيروح بتـ Check الـ Data signal يلاقي الـ out_aux low فيخلي الـ Status 0.

واول ما يخلص الـ out_aux بيبقي high الـ data enable بيبقي high يشوف الـ data signal، بما ان الـ out_aux بـ 1 فالـ or gate يدينا واحد فالـ status تكون 1 الـ هو Done.

بكده وفرنا كمان axi transaction.

Slide: Software Verification 2

زي ما قولنا اول حالة دي الـ latency طلعتي انها 442.

تاني phase:

بعد ما حجم الـ input والـ output صغرناه لـ 12 bit و عملنا الـ optimization بقت 59 cycle.

ثالث Phase:

بقينا بنبعث twiddle index بس ونـ read الـ twiddle factor من internal ROM واستخدمنا standard multiplier والـ optimizations الـ auto trigger و auto status.

الـ latency نزلت لـ 24 cycle، الـ butterfly بياخد 7 cycles والـ 17 الباقيين overhead.

(Note: انا جربت tests كتير عشاني اضمن انها مش معتمدة علي الـ inputs في حاجة).

Verification 2 & Slide: Software

دول snippets من الـ Results.

فعندي كام سؤال ممكن في الآخر عشاني الوقت.